

GLK Data Handling — Response

8 September 2026 | From: Backend Team

1. 0x0E Status vs 0x4E Sleep Stage

These two are **never mixed**. The system treats them as completely different data streams:

- **0x0E "status" (every 5 sec):** The device's real-time operational state — "in bed", "snoring", "out of bed", etc. "Light Sleep" at status=6 is just a device state label, not a sleep architecture classification. Stored in the field `glk_status`.
- **0x4E "sleep stage" (every ~1 min):** The actual sleep architecture — awake, light, deep, REM. Stored in a completely separate field `glk_sleep_stage`.

Each has its own parser, its own database field, and its own status mapping. The dashboard uses only `glk_sleep_stage` for the light/deep/rem/awake breakdown — it never confuses 0x0E status=6 with actual sleep staging.

No changes needed.

2. body_movement 237 During Snoring

The concern is valid — 237 during snoring doesn't make sense if it were a proportional "how much are they moving" index. Our real data confirms this: nearly half of all in-bed readings show values above 200, even during quiet sleep and snoring. This strongly suggests byte 10 is a **raw sensor amplitude** (snoring vibrations register as high values), not a linear activity index.

What we've done:

- The **raw 0-255 value is always stored as-is** — nothing is lost at storage.
- For display, we apply a **compressed curve**: values up to 230 show as low activity (0-4 on a 0-10 scale), only 230-255 shows as tossing/turning. So 237 during snoring displays as ~5.7 instead of the misleading 9.3 a straight-line scale would give.
- This curve is **display-only** — it doesn't change what's stored.
- **No alerts or notifications** are triggered by this field.

Pending: Need vendor clarification on what byte 10 actually represents before building anything further on it.

3. Store Raw, Act on Almost Nothing

Everything the device sends is **stored raw** in the database — heart rate, breathing rate, movement, battery, signal quality, sleep stages, raw hex frames. Nothing is thrown away.

The only field that **actively drives behavior** in the app is `in_bed` (is the person on the bed). That's it. Everything else is display-only:

- Heart rate / breathing rate — shown on the sleep screen, no alerts.
- Sleep stages — shown as "sleep stages as reported by the bed sensor." No scoring, no assessment.
- Internal flags (life abnormality, sleep apnea) — stored in DB but **never shown to families using those clinical terms**. Family-facing messages say "unusual reading from the bed sensor."

No changes needed.

4. Pi Write Pattern — SD Card Longevity

The bridge does **NOT** write data files to the SD card. All data goes out over the network as HTTP requests:

- **0x0E**: 1 HTTP request every 5 seconds — approximately 17,280 per day.
- **0x4E**: 1 HTTP request per frame (roughly per minute during sleep) — approximately 480 per night.

These go to the local Node server on the Pi and to the cloud backend — no data files are written to disk.

The **SD card wear risk is from system logs**, not from sensor data.

Recommended before 15-home rollout:

1. Reduce logging to warnings-only in production (~95% fewer disk writes).
2. Move log directories to RAM disk (tmpfs) so they never touch the SD card.
3. If offline buffering is needed, batch writes every 60 seconds to a single file.
4. Consider read-only SD card with RAM overlay (overlayroot) — standard for long-running Pi deployments.

5. 0x4E ACK

The bridge sends the required acknowledgment immediately for every 0x4E frame received. Format: 82 05 00 <seq> 4E 01 <checksum>. Live since the v24 bridge deployment — no device disconnection issues since.

Implemented and running.

6. REM Sleep Extraction

Fully working end-to-end:

- Pi reads the sleep stage byte from 0x4E frames (code 5 = REM).
- Backend stores it in the `glk_sleep_stage` field.
- Dashboard calculates duration per stage (light / deep / rem / awake).
- App shows the breakdown on the sleep detail screen. We display exactly what the device reports.

Implemented end-to-end.

Summary

#	Concern	Status	Next Step
1	0x0E vs 0x4E mixed	Already separate	None
2	body_movement meaning	Raw stored; compressed display curve	Vendor clarification on byte 10
3	Don't act on unvalidated fields	Only in_bed drives behavior	None
4	SD card wear	HTTP, not disk; logs are the risk	Reduce logging + RAM disk before rollout
5	0x4E ACK	Implemented and running	None
6	REM extraction	Working end-to-end	None